

# CS488 Final Project Proposal

## Simplified Grid-Based Sand Pile Simulation with Friction and Surface Reconstruction

Yutong Wu  
Student ID: 21020921

### Summary

For my final project, I plan to implement a simplified physics-based sand pile simulator. After receiving feedback on my first draft, I will focus only on sand rather than trying to simulate both snow and sand. The project will not attempt a full production-quality sand solver, but it will implement a small, understandable sand simulation based on a particle-grid representation, frictional flow, and surface reconstruction.

The final program will show sand particles falling into a small scene, interacting with a ground plane and a simple obstacle such as a ramp or box, transitioning between flowing and settled regions, and forming a visible pile. The project will be inspired by Zhu and Bridson's continuum view of sand, where sand is represented by particles for advection and surface tracking, but an auxiliary grid is used to compute spatial interactions such as boundary conditions and friction. My implementation will be a simplified version suitable for the existing CS488 framework.

The main technical focus is not merely to tune particle parameters until the result looks like sand. Instead, I will implement an explicit sand model with a grid-based density/velocity representation, frictional boundary response, a simplified internal friction/yield rule, and a reconstructed surface for rendering the pile. The demo will show the simulation under different friction settings and will include visual diagnostics for grid density, velocity, and flowing-versus-settled regions.

### Technical Outline

#### Representation

The simulation will use particles to carry sand mass, position, and velocity. Each particle will store:

- position and velocity,
- mass and radius,
- a cell index for grid insertion,
- a state flag indicating whether it is flowing or settled.

In addition to particles, the simulation will maintain a uniform Cartesian grid over the scene. Each grid cell will store accumulated mass, average velocity, occupancy/density, and a material state. The grid will be used as a computational structure rather than only as a collision acceleration structure. This is the main difference from the particle-only system in Assignment 3.

#### Particle-Grid Update

Each time step will follow a simple particle-grid-particle cycle:

1. Clear grid mass and velocity values.

2. Transfer particle mass and velocity to nearby grid cells using a simple weighting scheme.
3. Apply gravity and collision/friction corrections on the grid velocity field.
4. Apply a simplified internal friction/yield rule on grid cells.
5. Transfer corrected grid velocities back to particles.
6. Advect particles using the updated velocities.

This is not intended to be a complete FLIP or MPM solver. It is a simplified hybrid method designed to demonstrate the key graphics idea that particles are useful for advection and surface tracking, while a grid is useful for neighborhood and continuum-style calculations.

### **Friction and Yield Model**

I will implement two levels of friction. First, boundary friction will occur when sand contacts the ground or an obstacle. The particle or grid velocity will be decomposed into normal and tangential components. The normal component will be corrected to prevent penetration, and the tangential component will be damped according to a friction coefficient.

Second, I will implement a simplified internal friction/yield rule. For each occupied grid cell, I will estimate local motion using neighboring grid velocities. If the local shear-like motion is small relative to the cell's pressure/density proxy, the cell can be marked as settled or rigid-like. If the local slope or shear exceeds a threshold related to a friction angle, the cell will be allowed to flow. This is a simplified, visually motivated version of sand's flowing-versus-stable behavior. The goal is to demonstrate a physics-based rule, not to exactly reproduce a full Mohr-Coulomb solver.

### **Obstacle and Ground Interaction**

The scene will contain a ground plane and at least one simple obstacle. Possible obstacles include a ramp plane, an axis-aligned box, or a sphere. I will implement geometric collision tests for the chosen obstacles. When sand penetrates an obstacle, its position will be projected out and the velocity will be modified using normal correction and tangential friction. This should let sand fall onto an obstacle, slide down ramps, and accumulate near boundaries.

### **Pile Accumulation**

The simulation will include a settling rule. A particle or grid cell can become settled when its speed is small, it is supported by the ground or nearby dense cells, and the local slope is below a chosen angle-of-repose threshold. Settled sand will still be able to reactivate if neighboring material moves or if the local slope becomes too steep. This should make piles less jittery and allow the demo to show stable accumulation.

### **Surface Reconstruction and Rendering**

The sand particles will be converted into a renderable surface. I plan to use a height-field or density-grid reconstruction: particle mass will be accumulated into a 2D height grid over the ground, and the resulting heights will be converted into a triangle mesh with computed normals. This surface will be rendered with a sand-colored material. To show some grain-like detail without implementing a full granular light transport model, I will add procedural noise or color variation based on height, slope, or density.

The renderer will also include debugging views. These views may show cell density, velocity magnitude, or flowing versus settled regions. The debug views will make it easier to demonstrate that the grid and friction rules are actually affecting the simulation.

## Objectives

1. Implement a particle-to-grid transfer that accumulates sand mass and velocity into a uniform Cartesian grid each simulation step.
2. Implement a grid-to-particle velocity update so particles are advected by corrected grid velocities rather than only by independent particle motion.
3. Implement boundary friction for sand-ground and sand-obstacle contact by decomposing velocity into normal and tangential components and damping tangential motion.
4. Implement collision with at least one non-ground obstacle, such as a ramp, box, or sphere, including position correction and frictional velocity response.
5. Implement a simplified internal friction/yield rule that classifies grid cells as flowing or settled based on local density, velocity differences, and a friction-angle threshold.
6. Implement a settling/reactivation rule so sand can form stable piles but can begin moving again when local slope or neighboring motion exceeds a threshold.
7. Implement height-field or density-grid surface reconstruction from particles and render the reconstructed sand pile as a triangle mesh with computed normals.
8. Implement diagnostic visualization and measurement modes showing at least two of the following: grid density, velocity magnitude, flowing/settled state, or measured pile slope/angle of repose.

## Expected Final Output

The final output will be a small scene in which sand falls under gravity, flows over or around an obstacle, and accumulates into a visible pile. The demo video will show the particle-grid update, obstacle friction, settling behavior, reconstructed surface rendering, and diagnostic views. I will also include screenshots or short video segments comparing different friction-angle settings to show how the angle of repose affects the final pile shape.

## Scope and Limitations

This project will focus only on sand. I will not attempt snow simulation, since snow requires a more complex elasto-plastic model and would be too large in scope for this project. I also will not implement a full FLIP, MPM, or multi-scale granular renderer. The goal is to build a simplified but explicit physics-based sand simulation that is more principled than a purely heuristic particle system.

## References

- Yongning Zhu and Robert Bridson. *Animating Sand as a Fluid*. ACM SIGGRAPH 2005.

- Johannes Meng, Marios Papas, Ralf Habel, Carsten Dachsbacher, Steve Marschner, Markus Gross, and Wojciech Jarosz. *Multi-Scale Modeling and Rendering of Granular Materials*. ACM SIGGRAPH 2015.
- Alexey Stomakhin, Craig Schroeder, Lawrence Chai, Joseph Teran, and Andrew Selle. *A Material Point Method for Snow Simulation*. ACM SIGGRAPH 2013.